

Reading sales bills from the POS

MST Golf runs a membership system: members earn points on what they buy. To give points automatically, our system needs to **read the store's bills from the POS through an API**. This page lists what we need.

We only read

Our system calls your API. We never create, change or delete anything in the POS.

Every 15 minutes

We ask for the bills that are new or changed since our last check.

No webhooks

You don't need to send anything to us or call our system.

1. Access

Login	An API key (or OAuth 2.0 client credentials) that can only read bills, limited to MST Golf's store.
Format	HTTPS and JSON. Dates with a time zone (ISO 8601, e.g. <code>2026-10-15T18:42:10+07:00</code>). Money as numbers with 2 decimals.
IP address	Our system runs in the cloud and has no fixed IP, so please don't require an IP whitelist.
History	We need to read at least the last 13 months of bills once, when we start.
Not needed	Card numbers, or any customer data other than the member's phone number or member code typed on the bill.

2. Endpoints

Endpoint (example name)	What we use it for	Need
GET /bills?updated_since=...	Bills created or changed after a given time. This must include refunds, voids and bills that reached the server late (e.g. after the internet was down). Paged.	Required
GET /bills?from=...&to=...	Bills by sale date. Used once to load past sales, and to re-read a day if needed.	Required
GET /bills/{id}	One bill, to check when a customer asks about their points.	Nice to have
GET /products	Product brand and category, if bills don't include them.	Nice to have

Your field and endpoint names don't have to match ours — just tell us the real names in your API documentation.

What each bill needs to contain

Everything is required unless marked (nice) — nice to have.

3. Bill

bill_no	Unique, and never changes after the bill is closed. We use it to never give points twice.
type	Sale, refund or void.
ref_bill_no	For refunds and voids: the original sale's bill number.
sold_at, updated_at	When it was sold, and when it was last saved or changed.
status	Closed, on hold or cancelled. We only count closed bills.
branch, terminal	Which store and which till.
gross, discount, net	Total before discount, bill discount, amount paid. Say whether VAT is included.
member_ref	Member's phone or member code. A separate field is best; the bill remark also works (staff type <code>MSTMEMBER:0891112233</code>).
remark	The bill's note, in full.
payments (nice)	Payment methods and amounts.

4. Items on the bill

sku, name	Product code and name.
qty, unit_price	Quantity and price each.
line_discount, line_net	Discount on the item and the item's total.
brand, category (nice)	Lets the store exclude some products from points.
item_type (nice)	Marks gift cards, vouchers and services.
ref_line (nice)	For a partial refund: which item was returned.

To get started, please send

- Your full API documentation
- A test environment (sandbox) and a test API key
- One real example each: a sale, a refund, a void

5. Example — a sale with a member

For illustration only. Your format can differ.

```
{ "bill_no": "CIT1-20261015-0042", "type": "SALE", "status": "CLOSED",
  "branch": "CIT1", "terminal": "POS-01",
  "sold_at": "2026-10-15T18:42:10+07:00", "updated_at": "2026-10-15T18:42:12+07:00",
  "member_ref": "0891112233", "remark": "MSTMEMBER:0891112233",
  "gross": 21800.00, "discount": 900.00, "net": 20900.00, "vat_included": true,
  "lines": [
    { "sku": "QI35-D-10.5", "name": "Qi35 Driver", "brand": "TaylorMade", "category": "DRIVER",
      "qty": 1, "unit_price": 20900.00, "line_discount": 900.00, "line_net": 20000.00 },
    { "sku": "PV1-12", "name": "Pro V1", "brand": "Titleist", "category": "BALL",
      "qty": 1, "unit_price": 900.00, "line_discount": 0, "line_net": 900.00 }
  ] }
```

A refund looks the same with `"type": "REFUND"`, `"ref_bill_no"` pointing to the sale, and the returned items.